Лабораторна робота 10. Форми та валідація оформлення замовлення

Завдання та очікуваний результат

Замініть просте підтвердження на форму покупця: ім'я, телефон, спосіб отримання, умовна адреса та коментар. Замовлення повинно зберігатися тільки після перевірки даних.

1. Окрема модель введення `CheckoutInput`, а не редагування сутності БД напряму.
2. `EditForm`, компоненти введення, повідомлення біля полів і загальне зведення.
3. Стандартні анотації та правило «для доставки потрібна адреса».
4. Перевірка і в інтерфейсі, і в сервісі збереження.
5. Відображення введених даних у збереженому замовленні.

Продовжуйте власний проєкт із лабораторної 9. Не створюйте новий шаблон і не замінюйте свою тему демонстраційною. Перед початком збережіть попередній результат комітом або окремою копією.

Готовий контрольний варіант — папка `Lab10_CampusCafe`. У ній застосунок `CampusCafe`, перевірки `CampusCafe.Checks` та README. Демо є результатом саме цього етапу, а не початковим шаблоном для механічної заміни власного проєкту.

Межі навчального застосунку: оплати, автентифікації та розмежування доступу немає. Список замовлень спільний, а не захищений список «моїх» покупок. Запускайте демо локально та вводьте тільки вигадані дані. Публікація вихідного коду не означає готовність сервісу приймати реальних покупців.

Як працювати з кодом

У прикладі збережено назви `CampusCafe`, `Drink` і кавові опції. У своєму застосунку продовжуйте тему, обрану раніше: букети, фотодрук, подарункові набори, сендвічі чи інший каталог. Під час замін не втрачайте власні назви, формули та оформлення.

Повні лістинги показують узгоджений результат кроку. Перед додаванням коду визначте, де зберігається введене значення, хто його перевіряє і хто показує помилку. Для невеликих правок використано `diff`: рядок із `-` вилучається, із `+` додається без знака `+`; рядки без цих знаків — контекст. Заголовки `---`, `+++` і `@@` не копіюйте до програми. Для великих змін наведено повні версії «Було / Стало», для CSS — лише новий файл.

Перед запуском кожного прикладу спрогнозуйте результат для правильного і неправильного введення. Після запуску зіставте прогноз із поведінкою форми. Для своєї теми сформулюйте правило словами, а потім знайдіть місце його реалізації: атрибут властивості, перевірка кількох полів або серверний сервіс. Демо використовуйте для порівняння, зберігаючи власні моделі й оформлення.

Шляхи в лістингах відраховано від папки `CampusCafe`, де лежить `.csproj`. Це не вимога називати власний проєкт так само. Після зміни простору імен узгоджуйте його в усіх файлах через Rename редактора.

Після завершення кожного логічного етапу виконуйте `dotnet build`. Нові взаємозалежні класи спочатку додайте всі, а потім збирайте. Контрольні точки нижче вказують, коли очікується повністю працездатна версія.

1. Що саме перевіряє форма

Звичайний `@bind` змінює властивість, але сам не визначає, чи значення допустиме. `EditForm` створює або отримує `EditContext` — об'єкт, який знає модель, змінені поля та повідомлення перевірки. Значення зберігаються у моделі, а не в окремій копії всередині контексту.

```
Введення → CheckoutInput → EditContext + DataAnnotationsValidator
                              ↓ коректні дані
                          OnValidSubmit
                              ↓
                    PurchaseService перевіряє ще раз
                              ↓
                    PurchaseOrder → SQLite
```

У нашому Interactive Server перевірка C# виконується на сервері. Повідомлення біля поля не означає, що цей C# працює в браузері. Декларативна/програмна перевірка та клієнтська/серверна — різні класифікації.

Модель введення — тимчасова. Сутність замовлення — постійний запис. Не слід дозволяти формі змінювати `Id`, `CreatedUtc`, ціну або `RequestId`.

2. Модель введення та правила

`Required` перевіряє обов'язковість, `StringLength` — довжину, `RegularExpression` — формат. Телефон у демонстрації має формат `+380` і дев'ять цифр; це перевірка синтаксису, а не існування номера. Використовуйте лише вигаданий навчальний номер `+380000000000`.

`IValidatableObject` дозволяє перевірити залежність між полями. Адреса потрібна лише для доставки. `nameof(Address)` пов'язує повідомлення саме з цим полем. За наявності помилок окремих властивостей об'єктна перевірка може виконатися лише після їх виправлення — це нормальна послідовність Data Annotations.

У прикладі модель плоска: не потрібен експериментальний валідатор графа об'єктів. Для вкладених моделей у .NET 10 існує окремий механізм, але він не потрібний для цього завдання.

Адаптуйте зараз: спосіб отримання й умову власної теми. Наприклад, для друку — вимога тексту, коли вибрано персоналізацію. Не додавайте поля без практичного призначення.

Файл `Models/CheckoutInput.cs`

Розбір властивостей. `CheckoutInput` зберігає те, що користувач редагує зараз. Його властивості змінюватимуть компоненти введення. `PurchaseOrder` із попередньої роботи буде створено пізніше, коли перевірки пройдено. Це дозволяє не надавати формі доступ до службових полів збереженого запису.

`FulfillmentKind` — перелік допустимих варіантів. Використання `Pickup` і `Delivery` замість довільних рядків зменшує ризик помилок у назвах. `[EnumDataType]` перевіряє допустимість значення, адже числове значення enum теоретично може не відповідати оголошеним елементам.

У регулярному виразі `^` позначає початок рядка, `\+` — буквальний плюс, `[0-9]{9}` — рівно дев'ять цифр, `$` — кінець рядка. Префікс `@` перед C#-рядком дозволяє не подвоювати зворотний слеш. Отже, додаткові пробіли чи літери не пройдуть перевірку.

Чому адреса без `[Required]`. Постійний атрибут вимагав би адресу і для самовивозу. Залежне правило міститься в `Validate`: воно читає одночасно `Fulfillment` та `Address`. `yield return` додає результат перевірки до послідовності помилок; якщо умова не спрацювала, ця помилка не створюється. `new[] { nameof(Address) }` прив'язує повідомлення до поля адреси, щоб його можна було показати поруч із input.

`Trim()` прибирає крайні пробіли для перевірки змісту. Без цього рядок із кількох пробілів міг би задовольнити просту вимогу довжини. Сам виклик `Trim()` тут не змінює властивість моделі: збереження очищеного значення буде виконано сервісом.

Перед реалізацією: складіть чотири приклади: коректний самовивіз, коректна доставка, доставка без адреси, некоректний телефон. Для кожного вкажіть, який атрибут або рядок `Validate` має його прийняти чи відхилити. Спочатку виправляйте помилки властивостей, потім перевіряйте залежне правило.

Створіть модель введення з правилами своєї теми:

```
using System.ComponentModel.DataAnnotations;

namespace CampusCafe.Models;

public enum FulfillmentKind { Pickup, Delivery }

public class CheckoutInput : IValidatableObject
{
    [Required(ErrorMessage = "Укажіть ім'я.")]
    [StringLength(60, MinimumLength = 2, ErrorMessage = "Ім'я має містити від 2 до 60
символів.")]
    public string Name { get; set; } = "";

    [Required(ErrorMessage = "Укажіть телефон.")]
    [RegularExpression(@"^\+380[0-9]{9}$", ErrorMessage = "Формат телефону: +380 і дев'ять
цифр, без пробілів.")]
    public string Phone { get; set; } = "";

    [EnumDataType(typeof(FulfillmentKind), ErrorMessage = "Оберіть допустимий спосіб
отримання.")]
```

```
    public FulfillmentKind Fulfillment { get; set; } = FulfillmentKind.Pickup;

    [StringLength(200, ErrorMessage = "Адреса має містити не більш як 200 символів.")]
    public string Address { get; set; } = "";

    [StringLength(500, ErrorMessage = "Коментар має містити не більш як 500 символів.")]
    public string Comment { get; set; } = "";

    public IEnumerable<ValidationResult> Validate(ValidationContext validationContext)
    {
        if ((Name ?? "").Trim().Length < 2)
            yield return new ValidationResult("Укажіть щонайменше два символи імені без
крайніх пробілів.", new[] { nameof(Name) });
        if (Fulfillment == FulfillmentKind.Delivery && (Address ?? "").Trim().Length < 5)
            yield return new ValidationResult("Для доставки вкажіть адресу щонайменше з 5
символів.", new[] { nameof(Address) });
    }
}
```

3. Компонент полів покупця

`InputText` та `InputSelect` працюють усередині `EditForm` і повідомляють його контекст про зміну поля. Для них використовуємо `@bind-Value`, а не HTML-варіант `@bind`.

`ValidationMessage For` містить вираз доступу до властивості. Завдяки цьому компонент знає, біля якого поля показувати помилку. Коли користувач обирає самовивіз, адреса зникає з розмітки; сервіс додатково не збереже старий текст адреси для самовивозу.

`@ref` отримує посилання на компонент `InputText`. Після першого інтерактивного рендерингу його `Element` уже існує, тому можна встановити фокус. Робити це під час конструктора чи `OnInitialized` зарано. `firstRender` не дає повертати фокус до імені після кожної зміни поля.

Файл `Components/Shared/CustomerFields.razor`

Як текст потрапляє до моделі. Батьківська сторінка передає об'єкт через `Input="input"`. Дочірній компонент отримує посилання на цей самий об'єкт у властивість `[Parameter] Input`. `@bind-Value="Input.Name"` читає та змінює його `Name`. Копія моделі всередині компонента не створюється, а збереження до SQLite тут узагалі не виконується.

Це свідомо обрана схема: дочірній компонент редагує поля спільної моделі форми. Він не повинен підміняти `Input` новим об'єктом. Контекст перевірки залишається у батьківської сторінки та надходить до компонентів введення каскадно через `EditForm`.

Що робить кожен вид розмітки:

- `label for` і відповідний `id` пов'язують підпис із полем, зокрема для доступності та переходу фокуса.
- `InputText`, `InputSelect` і `InputTextArea` змінюють властивості та повідомляють контекст про редагування. У типовому `InputText` значення підтверджується на `onchange`, а не після кожної літери.

- `ValidationMessage For="@(() => Input.Name)"` указує, помилки якої властивості показати. Сам вираз не перевіряє ім’я, а ідентифікує поле.
- `if` для доставки додає або прибирає поле адреси з інтерфейсу. Він не стирає `Input.Address` з пам’яті.
- `@ref` надає посилання на компонент введення. `nameInput?.Element is { } element` перевіряє наявність його HTML-елемента перед `FocusAsync`.

Контроль після підключення форми в кроці 5: введіть адресу, перемкніться на самовивіз і знову на доставку. Поясніть, чому текст може залишитися, хоча поле на деякий час зникало. Потім перевірте, що перше відображення ставить фокус на ім’я, а подальше редагування його не перехоплює.

Створіть компонент полів. На цьому кроці він ще не є самостійною сторінкою й не повинен містити власний `EditForm`:

```
<div class="form-fields">
    <label for="customer-name">Ім’я</label>
    <InputText id="customer-name" @ref="nameInput" @bind-Value="Input.Name"
autocomplete="off" />
    <ValidationMessage For="@(() => Input.Name)" />
    <label for="customer-phone">Телефон</label>
    <InputText id="customer-phone" @bind-Value="Input.Phone" type="tel" placeholder="+380…"
autocomplete="off" />
    <ValidationMessage For="@(() => Input.Phone)" />
    <label for="fulfillment">Спосіб отримання</label>
    <InputSelect id="fulfillment" @bind-Value="Input.Fulfillment">
        <option value="@FulfillmentKind.Pickup">Самовивіз</option>
        <option value="@FulfillmentKind.Delivery">Доставка</option>
    </InputSelect>
    <ValidationMessage For="@(() => Input.Fulfillment)" />
    @if (Input.Fulfillment == FulfillmentKind.Delivery)
    {
        <label for="customer-address">Адреса доставки</label>
        <InputText id="customer-address" @bind-Value="Input.Address" autocomplete="off" />
        <ValidationMessage For="@(() => Input.Address)" />
    }
    <label for="customer-comment">Коментар</label>
    <InputTextArea id="customer-comment" @bind-Value="Input.Comment" rows="3" />
    <ValidationMessage For="@(() => Input.Comment)" />
</div>

@code {
    [Parameter, EditorRequired] public CheckoutInput Input { get; set; } = default!;
    private InputText? nameInput;
    protected override async Task OnAfterRenderAsync(bool firstRender)
    {
        if (firstRender && nameInput?.Element is { } element)
            await element.FocusAsync();
    }
}
```

4. Незалежна перевірка в сервісі

Перед зміною сторінки оновіть контракт сервісу: тепер він вимагає `CheckoutInput`. До оновлення Checkout і тестів компілятор вказуватиме на старі виклики з двома аргументами — це очікувана проміжна ситуація.

`Validator.TryValidateObject(..., validateAllProperties: true)` запускає правила моделі також поза UI. `OnValidSubmit` не є заміною цієї перевірки: сервіс можуть викликати тест, інша сторінка або майбутній API.

Стовпці для покупця вже створені в лабораторній 9, тому схема БД не змінюється і видаляти `orders.db` не потрібно. Старі замовлення без покупця залишаються допустимою історією попереднього етапу.

Адаптуйте зараз: зіставлення полів форми зі збереженим замовленням. Не замінюйте серверну перевірку ціни просто довірою до значення з кошика.

Файл `Services/PurchaseService.cs`

Як змінюється контракт. Новий третій аргумент `CheckoutInput input` означає, що оформлення тепер вимагає дані покупця. Знайдіть усі виклики `PlaceAsync` у своєму проєкті та тестах. Для коректної спроби кожен із них має передати заповнену модель. Не створюйте порожній `CheckoutInput` лише для усунення помилки компіляції: сервіс правильно відхилить його.

На початку методу `ThrowIfNull` відхиляє відсутній об'єкт. `ValidationContext` описує об'єкт для валідатора, список `errors` збирає помилки, а `validateAllProperties: true` включає перевірку анотацій властивостей. Якщо правила порушено, виняток виникає до створення нового замовлення. Повідомлення об'єднуються для показу загальної помилки на сторінці.

Другий фрагмент змін переносить дозволені дані з тимчасової моделі до постійної сутності. Назви не зобов'язані збігатися: `input.Name` стає `CustomerName`, `input.Address` — `DeliveryAddress`. Для самовивозу зберігається порожня адреса незалежно від старого тексту в моделі. Зверніть увагу, що `Fulfillment` тут не має власного стовпця: спосіб отримання у цій навчальній схемі відображається через наявність адреси.

Що залишається з попередньої роботи. Перевірки складу, повторний розрахунок ціни та захист від дубля за `RequestId` продовжують діяти. Правильне ім'я не робить підмінену ціну допустимою. Форма і сервіс перевіряють різні аспекти однієї операції, хоча частину правил введення використовують спільно.

Контроль незалежності: виклик сервісу з порожньою моделлю в автоматичній перевірці повинен завершитися відмовою без відкриття браузера. Це показує, що правило не залежить від того, чи була на екрані форма.

Внесіть локальні зміни:

```
--- a/Services/PurchaseService.cs
+++ b/Services/PurchaseService.cs
@@ -1,4 +1,5 @@
```

```
 using CampusCafe.Data;
+using System.ComponentModel.DataAnnotations;
 using CampusCafe.Models;
 using Microsoft.EntityFrameworkCore;

@@ -8,8 +9,13 @@
     IDbContextFactory<OrderDbContext> factory,
     CatalogService catalog)
 {
-    public async Task<int> PlaceAsync(IReadOnlyList<OrderLine> lines, Guid requestId)
+    public async Task<int> PlaceAsync(IReadOnlyList<OrderLine> lines, Guid requestId,
CheckoutInput input)
     {
+        ArgumentNullException.ThrowIfNull(input);
+        // Сервіс перевіряє дані незалежно від наявності EditForm.
+        var errors = new List<ValidationResult>();
+        if (!Validator.TryValidateObject(input, new ValidationContext(input), errors,
validateAllProperties: true))
+            throw new ArgumentException(string.Join(" ", errors.Select(e =>
e.ErrorMessage)));
         if (requestId == Guid.Empty)
             throw new ArgumentException("Відсутній ідентифікатор операції.");
         await using var db = await factory.CreateDbContextAsync();
@@ -19,7 +25,15 @@
             throw new ArgumentException("Замовлення має містити від 1 до 50 позицій.");

         var products = (await catalog.GetDrinksAsync()).ToDictionary(p => p.Id);
-        var order = new PurchaseOrder { RequestId = requestId, CreatedUtc =
DateTime.UtcNow };
+        var order = new PurchaseOrder
+        {
+            RequestId = requestId,
+            CreatedUtc = DateTime.UtcNow,
+            CustomerName = input.Name.Trim(),
+            Phone = input.Phone,
+            DeliveryAddress = input.Fulfillment == FulfillmentKind.Delivery ?
input.Address.Trim() : "",
+            Comment = (input.Comment ?? "").Trim()
+        };
         foreach (var line in lines)
         {
             if (!products.TryGetValue(line.DrinkId, out var product))
```

5. EditForm і керування надсиланням

У формі задаємо **або** `Model`, **або** `EditContext`, а не обидва одночасно. Тут контекст створюється один раз у `OnInitialized` і пов'язаний із тим самим екземпляром `input`.

`OnValidSubmit` запускає збереження, `OnInvalidSubmit` показує коротку вказівку. `OnSubmit` не додаємо разом із ними: це інший режим, у якому перевірку запускають вручну через `context.Validate()`.

`FormName` дає формі ім'я, але не вмикає валідацію. У нашому інтерактивному режимі подія форми обробляється Blazor; не потрібно додавати Razor Pages `OnPost` чи `[BindProperty]`.

`OnFieldChanged` тут лише повідомляє про редагування, а не перевіряє всю форму після кожного символу. Підписку прибираємо у `Dispose`. Не замінюйте модель новим об'єктом, залишивши старий `EditContext`.

Зміна приходить від вкладеного компонента введення, тому `InvokeAsync(StateHasChanged)` оновлює також повідомлення батьківської сторінки. Інакше текст про попередню помилку міг би залишатися видимим до наступного надсилання, хоча поле `error` уже очищене.

Кнопка доступна і для невалідної форми, щоб користувач міг отримати пояснення. На час збереження блокуємо кнопку й поля. Кошик очищується тільки після успіху.

Файл `Components/Pages/Checkout.razor`

Порівняння ролей старої та нової сторінки. Попередня версія мала одну команду: зберегти поточний кошик. Нова версія спочатку збирає дані покупця та перевіряє їх. Маршрут, сервіс збереження, очищення кошика після успіху й перехід до деталей залишаються.

Перед заміною знайдіть у версії «Стало» такі частини:

Частина	Що забезпечує
`input = new()`	Один екземпляр даних, які редагує користувач
`new EditContext(input)`	Облік змін та повідомлень для цього екземпляра
`DataAnnotationsValidator`	Підключення правил `CheckoutInput` до перевірки форми
`CustomerFields Input="input"`	Відображення полів без створення другої моделі
`ValidationSummary`	Загальний перелік помилок перевірки
`OnValidSubmit`	Запуск оформлення після успішної перевірки
`OnInvalidSubmit`	Загальна вказівка користувачу без виклику збереження
`fieldset disabled`	Блокування редагування полів на час поточної операції

Прохід одного натискання. Кнопка `type="submit"` надсилає форму до механізму `EditForm`. Контекст запитує перевірку, валідатор читає правила моделі. За наявності помилок викликається `ShowInvalid`, а за їх відсутності — `PlaceOrderAsync`. Усередині останнього сервіс повторно перевіряє дані, зберігає замовлення і повертає номер. Лише після цього кошик очищується та виконується перехід.

Підписка на зміни. `OnFieldChanged += FieldChanged` означає «повідомляй цей метод, коли компонент введення повідомив про зміну поля». `IsModified()` показує, чи відмічені поля як редаговані, а не чи форма правильна. Це також не точне порівняння поточного тексту з початковим: повернення старого значення не обов'язково скидає ознаку модифікації. `error = null` прибирає попереднє загальне повідомлення, але не підмінює перевірку полів.

`@implements IDisposable` дозволяє Blazor викликати `Dispose` при видаленні компонента. Рядок із `-=` прибирає саме ту підписку, яку раніше додали через `+=`. Це правило корисне і для сервісів, які живуть довше за сторінку.

Вправа під час заміни: до запуску поясніть, що зламається, якщо передати в `CustomerFields` інший `new CheckoutInput()`, а `EditContext` залишити на старому. Після реалізації перевірте помилкове введення, виправлення та успішне оформлення. Не достатньо перевірити лише один правильний сценарій.

Було — повний вміст попереднього етапу:

```
@page "/checkout"
@inject OrderState Cart
@inject PurchaseService Purchases
@inject NavigationManager Navigation
@inject ILogger<Checkout> Logger

<PageTitle>Перевірка замовлення</PageTitle>
<h1>Перевірка замовлення</h1>
<p>Навчальне замовлення без оплати. Форму покупця додамо в наступній роботі.</p>
@if (Cart.Items.Count == 0)
{
    <p>Кошик порожній. <a href="/">Повернутися до каталогу</a></p>
}
else
{
    <div class="panel">
        <OrderReview Items="Cart.Items" />
        @if (error is not null) { <p role="alert" class="validation-message">@error</p> }
        <button type="button" disabled="@isSubmitting" @onclick="PlaceOrderAsync">
            @(isSubmitting ? "Збереження…" : "Зберегти замовлення")
        </button>
        <a class="back-link" href="/">Повернутися до каталогу</a>
    </div>
}

@code {
    private bool isSubmitting;
    private string? error;
    private readonly Guid requestId = Guid.NewGuid();

    private async Task PlaceOrderAsync()
    {
        if (isSubmitting) return;
        isSubmitting = true;
        error = null;
        try
        {
            var id = await Purchases.PlaceAsync(Cart.Items.ToArray(), requestId);
            Cart.Clear();
            Navigation.NavigateTo($"/orders/{id}");
        }
        catch (ArgumentException ex) { error = ex.Message; }
        catch (InvalidOperationException ex) { error = ex.Message; }
        catch (Exception ex)
```

```
        {
            Logger.LogError(ex, "Помилка збереження замовлення");
            error = "Не вдалося зберегти замовлення. Кошик не очищено. Спробуйте ще раз.";
        }
        finally { isSubmitting = false; }
    }
}
```

Стало — повний вміст після заміни:

```
@page "/checkout"
@implements IDisposable
@inject OrderState Cart
@inject PurchaseService Purchases
@inject NavigationManager Navigation
@inject ILogger<Checkout> Logger

<PageTitle>Оформлення замовлення</PageTitle>
<h1>Оформлення замовлення</h1>
<p>Використовуйте лише вигадані навчальні дані. Оплата не проводиться.</p>
@if (Cart.Items.Count == 0)
{
    <p>Кошик порожній. <a href="/">Повернутися до каталогу</a></p>
}
else
{
    <EditForm EditContext="editContext" OnValidSubmit="PlaceOrderAsync"
              OnInvalidSubmit="ShowInvalid" FormName="checkout">
        <DataAnnotationsValidator />
        <div class="checkout-grid">
            <section class="panel">
                <h2>Ваш вибір</h2>
                <OrderReview Items="Cart.Items" />
            </section>
            <section class="panel">
                <h2>Дані покупця</h2>
                <fieldset disabled="@isSubmitting">
                    <CustomerFields Input="input" />
                </fieldset>
                <ValidationSummary />
                @if (error is not null) { <p role="alert" class="validation-
message">@error</p> }
                <p class="form-hint">@(hasChanges ? "Ви змінили дані форми." : "Заповніть
форму.")</p>
                <button type="submit" disabled="@isSubmitting">
                    @(isSubmitting ? "Збереження…" : "Підтвердити замовлення")
                </button>
            </section>
        </div>
    </EditForm>
}

@code {
    private readonly CheckoutInput input = new();
    private EditContext editContext = default!;
    private bool hasChanges;
```

```
    private bool isSubmitting;
    private string? error;
    private readonly Guid requestId = Guid.NewGuid();

    protected override void OnInitialized()
    {
        editContext = new EditContext(input);
        editContext.OnFieldChanged += FieldChanged;
    }

    private void FieldChanged(object? sender, FieldChangedEventArgs args)
    {
        hasChanges = editContext.IsModified();
        error = null;
        _ = InvokeAsync(StateHasChanged);
    }
    private void ShowInvalid(EditContext _) => error = "Виправте позначені поля та
повторіть спробу.";

    private async Task PlaceOrderAsync()
    {
        if (isSubmitting) return;
        isSubmitting = true;
        error = null;
        try
        {
            var id = await Purchases.PlaceAsync(Cart.Items.ToArray(), requestId, input);
            Cart.Clear();
            Navigation.NavigateTo($"/orders/{id}");
        }
        catch (ArgumentException ex) { error = ex.Message; }
        catch (InvalidOperationException ex) { error = ex.Message; }
        catch (Exception ex)
        {
            Logger.LogError(ex, "Помилка збереження замовлення");
            error = "Не вдалося зберегти замовлення. Кошик не очищено. Спробуйте ще раз.";
        }
        finally { isSubmitting = false; }
    }
    public void Dispose() => editContext.OnFieldChanged -= FieldChanged;
}
```

6. Перегляд результату й оформлення

У деталях показуємо дані покупця, якщо їх було введено. Це дозволяє переглядати і старі записи лабораторної 9. Razor кодує текст за замовчуванням: коментар не потрібно перетворювати на `MarkupString` або вставляти через `innerHTML`.

Стилі форми додаються до `orders.css`. Основний каталог, фото й кольори власної теми залишаються без змін.

Файл `Components/Pages/OrderDetails.razor`

Відображення постійних даних. Ця сторінка читає `order` із БД, а не поточний `input` із форми. Тому після перезавантаження вона повинна показати той самий результат. Перевірка `CustomerName` відділяє старі записи лабораторної 9 від нових. Вибір «Самовивіз» за порожньою адресою відповідає правилу збереження в сервісі.

Умовне відображення коментаря прибирає порожній абзац. Запис `@order.Comment` виводить текст із HTML-кодуванням. Для перевірки введіть нешкідливий текст `<b>подарунок</b>`: він має відображатися як текст із тегами, а не перетворюватися на HTML-розмітку.

Контроль: відкрийте нове замовлення в іншій вкладці та старе замовлення без покупця. Це перевіряє і збереження, і сумісність з попереднім етапом.

Внесіть локальні зміни:

```
--- a/Components/Pages/OrderDetails.razor
+++ b/Components/Pages/OrderDetails.razor
@@ -23,6 +23,13 @@
         </ul>
         <p class="review-total">Разом: <MoneyLabel Amount="order.Total" /></p>
         <p>Замовлення збережено. Оплата не проводилася.</p>
+        @if (!string.IsNullOrEmpty(order.CustomerName))
+        {
+            <h2>Отримання</h2>
+            <p>@order.CustomerName · @order.Phone</p>
+            <p>@(string.IsNullOrEmpty(order.DeliveryAddress) ? "Самовивіз" :
order.DeliveryAddress)</p>
+            @if (!string.IsNullOrEmpty(order.Comment)) { <p>@order.Comment</p> }
+        }
     </article>
 }
 <a class="back-link" href="/orders">До списку замовлень</a>
```

Файл `wwwroot/orders.css`

Що змінилося в оформленні. `.checkout-grid` створює дві колонки: перегляд кошика та дані покупця. Медіазапит переводить їх в одну колонку на вузькому екрані. `.form-fields` впорядковує підписи й поля, `font: inherit` узгоджує їх зі шрифтом сайту. `fieldset` групує поля, а його логічне блокування задається Razor-атрибутом, не стилем.

Збережені правила `.panel`, `.review-lines` та `.orders-list` потрібні сторінкам із попередньої роботи. Видаливши їх заради нової форми, ви зіпсуєте оформлення списку й деталей. Змінюйте кольори й відступи під власну тему, але перевіряйте всі три сторінки.

Замініть вміст файла. Нижче наведено лише нову повну версію; власну палітру з `app.css` збережіть.

```
.panel { background:#fff; border-radius:8px; padding:24px; margin:20px 0; box-shadow:0 2px
7px #0002; }
.review-lines { list-style:none; padding:0; }
```

```
.review-lines li { display:flex; flex-wrap:wrap; justify-content:space-between; gap:12px;
padding:14px 0; border-bottom:1px solid #ddd; }
.review-lines li span { color:#555; }
.review-total { font-size:1.3rem; font-weight:700; }
.orders-list { max-width:850px; }
.checkout-link { display:block; margin-top:14px; text-align:center; }
.back-link { display:inline-block; margin:18px; }
button:disabled { opacity:.55; cursor:not-allowed; }
nav { flex-wrap:wrap; }
.checkout-grid { display:grid; grid-template-columns:1fr 1fr; gap:24px; }
.form-fields { display:grid; gap:8px; }
.form-fields label { font-weight:700; margin-top:12px; }
.form-fields input,.form-fields textarea { width:100%; padding:10px; border:1px solid #999;
border-radius:4px; font:inherit; }
fieldset { border:0; padding:0; margin:0; min-width:0; }
.form-hint { color:#555; font-size:.9rem; }
@media (max-width:760px) { .checkout-grid { grid-template-columns:1fr; } }
```

7. Перевірки та самостійна частина

Перевірка	Очікуваний результат
Надіслати порожню форму	Помилки імені й телефону; запис не створено
Телефон `123`	Помилка формату
Коректні ім'я й телефон, самовивіз	Адреса не потрібна
Обрати доставку без адреси	Адресна помилка, збереження заблоковано
Ввести коректну адресу	Замовлення збережено з адресою
Коментар понад 500 символів	Помилка довжини
Повернутися до деталей замовлення, створеного в лабораторній 9	Сторінка працює без даних покупця
Прямий виклик сервісу з порожньою моделлю в тесті	Відхилення без участі EditForm

Самостійно: реалізуйте одне власне залежне правило і покажіть для нього коректний та некоректний приклади. Узгодьте правило, тексти помилок і серверне збереження. Якщо додаєте новий стовпець до сутності БД, це вже зміна схеми; для основного завдання достатньо існуючих полів.

Поясніть, чому `StringLength` не доводить правдивість імені, а правильний формат телефону не гарантує можливість доставки. Валідація — це визначені програмою правила, а не перевірка всіх обставин реального світу.

Додатково: порівняйте `OnValidSubmit` з `OnSubmit` і ручним `Validate()` в окремій копії сторінки. Не залишайте обидва підходи одночасно у фінальній формі. У другому варіанті поясніть, який саме рядок зупинить збереження, якщо `Validate()` поверне `false`. Перевірте це порожньою формою, а потім поверніть основний варіант.

Структура результату

```
CampusCafe/
  Components/App.razor
  Components/Layout/MainLayout.razor
  Components/Layout/MainLayout.razor.css
  Components/Layout/ReconnectModal.razor
  Components/Layout/ReconnectModal.razor.css
  Components/Layout/ReconnectModal.razor.js
  Components/Pages/About.razor
  Components/Pages/Checkout.razor
  Components/Pages/Error.razor
  Components/Pages/Home.razor
  Components/Pages/NotFound.razor
  Components/Pages/OrderDetails.razor
  Components/Pages/Orders.razor
  Components/Routes.razor
  Components/Shared/CartSummary.razor
  Components/Shared/ConfigureDrinkDialog.razor
  Components/Shared/CustomerFields.razor
  Components/Shared/DrinkCard.razor
  Components/Shared/MoneyLabel.razor
  Components/Shared/OrderReview.razor
  Components/_Imports.razor
  Data/CafeDbContext.cs
  Data/OrderDbContext.cs
  Data/SeedData.cs
  Models/CheckoutInput.cs
  Models/Drink.cs
  Models/DrinkDraft.cs
  Models/OrderLine.cs
  Models/PurchaseOrder.cs
  Models/ShopSettings.cs
  Program.cs
  Services/CatalogService.cs
  Services/OrderState.cs
  Services/PurchaseService.cs
  wwwroot/app.css
  wwwroot/orders.css
```

Підсумок і запитання для захисту

`CheckoutInput` містить введення й правила; `CustomerFields` — поля та повідомлення; `Checkout` — контекст і надсилання; `PurchaseService` — незалежну перевірку й запис. `DataAnnotationsValidator` підключає анотації до форми, `ValidationMessage` показує конкретну помилку, `ValidationSummary` — зведення.

Нові файли: `Models/CheckoutInput.cs`, `Shared/CustomerFields.razor`. Змінені: `Checkout.razor`, `PurchaseService.cs`, `OrderDetails.razor`, `orders.css`. Схема SQLite та попередні дані не змінюються.

Запитання: чому не можна передати одночасно `Model` і `EditContext`? Коли потрібен `OnSubmit`? Чому приховане поле не є автоматично очищеним? Навіщо сервіс перевіряє форму, якщо вже був `OnValidSubmit`?

Запуск і автоматичні перевірки

Із кореня готового комплекту:

```
dotnet run --project CampusCafe --urls http://localhost:5127
```

Для власного проєкту з його папки достатньо `dotnet run --urls http://localhost:5127`. Не запускайте кілька етапів на одному порту. Зупинка — `Ctrl+C`.

Окремо запустіть перевірки демо:

```
dotnet run --project CampusCafe.Checks
```

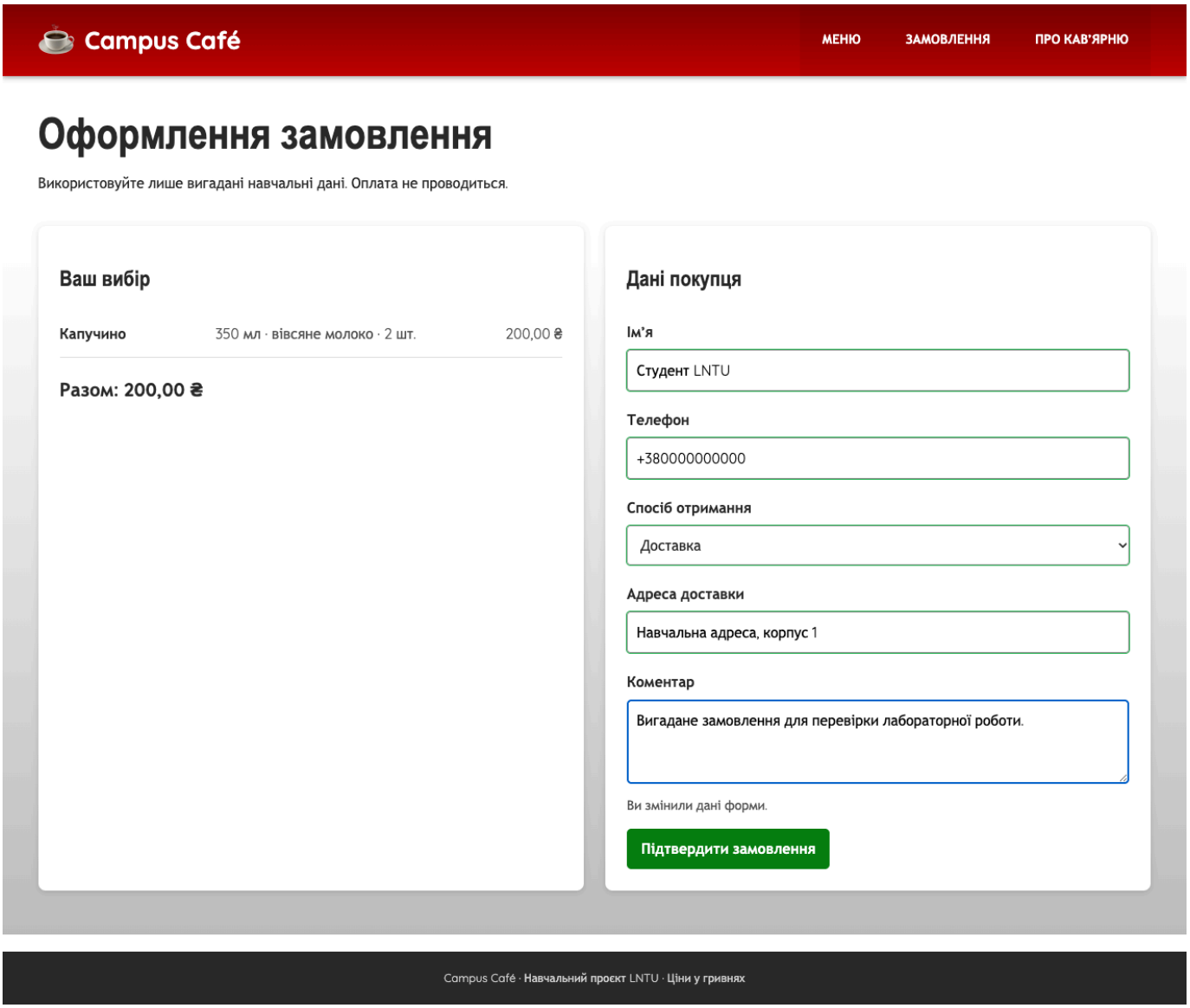
Тести працюють з окремими SQLite-базами в пам'яті й не змінюють робочі файли. Це перевірки C# і сховища, а не заміна ручного тестування браузера. Проєкт перевірок можна взяти з комплекту поточного етапу; для власних моделей адаптуйте типи, назви й контрольні ціни. Не запускайте старі тести зі старими сигнатурами сервісів.

Для здавання підготуйте вихідний код без `bin`, `obj` і персональних даних, короткий README, результати контрольних сценаріїв та пояснення власної тематичної адаптації. Збережіть окремий працездатний знімок цього етапу.

Наступний етап — лабораторна 11.

Джерела для поглиблення

- Форми.
- Валідація.
- Прив'язування.



Короткий підсумок роботи

Що реалізовано. Просте підтвердження замовлення доповнено формою покупця: ім'ям, телефоном, способом отримання, адресою для доставки та коментарем. Користувач отримує пояснення помилок, а коректні дані зберігаються разом із замовленням і відображаються на сторінці деталей.

Як це працює:

- **Модель введення.** `CheckoutInput` зберігає редаговані значення окремо від сутності БД. Атрибути Data Annotations перевіряють обов'язковість, довжину та формат, а `IValidatableObject` реалізує залежне правило: для доставки потрібна адреса.
- **Поля та прив'язування.** `CustomerFields` отримує ту саму модель, з якою працює сторінка. `@bind-Value` змінює її властивості, а `ValidationMessage` показує помилки біля відповідних полів.
- **Керування формою.** `EditContext` відстежує редагування й повідомлення перевірки. `EditForm` разом із `DataAnnotationsValidator` запускає валідацію: `OnValidSubmit` викликає збереження, а `OnInvalidSubmit` просить виправити введення.

- **Незалежна серверна перевірка.** `PurchaseService` повторно перевіряє модель і переносить дозволені дані до `PurchaseOrder`. Для самовивозу стара адреса з моделі не зберігається. Перевірки складу, ціни й повторного оформлення з попередньої роботи залишаються чинними.
- **Поведінка інтерфейсу.** Під час збереження поля й кнопка блокуються, після успіху кошик очищується. Фокус установлюється після першого інтерактивного рендерингу, а підписка на зміни полів прибирається в `Dispose`.

Головне запам'ятати: прив'язування змінює значення, валідація перевіряє їх, а сервіс зберігає результат. Це окремі дії. Правильний формат введення ще не гарантує правдивість даних або успішний запис до БД.

Тестові питання для самоперевірки

У кожному питанні оберіть **одну правильну відповідь**. Спочатку запишіть власні відповіді, потім перевірте їх за ключами нижче. Для помилкової відповіді простежте шлях даних від поля форми до сервісу.

1. Навіщо використовується окрема модель `CheckoutInput`?

- А. Щоб автоматично створювати окрему таблицю для кожного поля форми.
- Б. Щоб зберігати помилки замість введених значень.
- В. Щоб відокремити редаговані дані й правила введення від постійної сутності замовлення.
- Г. Щоб не перевіряти дані в сервісі.

2. Що зберігає та відстежує `EditContext`?

- А. Посилання на модель, стан редагування полів і пов'язані повідомлення перевірки.
- Б. Незалежну копію всіх значень, яку не потрібно пов'язувати з моделлю.
- В. Лише адресу сторінки для переходу після збереження.
- Г. Лише налаштування підключення до SQLite.

3. Який запис правильно підключає явно створений контекст до `EditForm`?

- А. Одночасно `Model="input"` і `EditContext="editContext"`.
- Б. Лише `FormName="checkout"`, без моделі або контексту.
- В. Лише `OnValidSubmit`, без моделі або контексту.
- Г. `EditContext="editContext"`, без одночасного встановлення `Model`.

4. Що робить `@bind-Value="Input.Name"` у `CustomerFields`?

- А. Зберігає ім'я безпосередньо в SQLite після кожної зміни.
- Б. Читає та змінює властивість переданого об'єкта моделі.
- В. Створює новий `CheckoutInput` при кожному рендерингу.
- Г. Викликає `OnPost` із Razor Pages.

5. Коли `EditForm` викликає `OnValidSubmit` у цьому прикладі?

- А. Щойно користувач відкрив сторінку.
- Б. Після будь-якої зміни тексту, незалежно від правил.
- В. Після надсилання форми, якщо її перевірка завершилася без помилок.
- Г. Лише після успішного запису до БД.

6. Чому правило обов'язкової адреси реалізовано через `IValidatableObject`?

- А. Воно залежить від поєднання способу отримання та значення адреси.
- Б. Атрибут `[Required]` узагалі не підтримує рядки.
- В. `InputText` не може редагувати адресу.
- Г. Будь-яку перевірку довжини потрібно писати лише вручну.

7. Користувач увів адресу, потім вибрав самовивіз. Що відбувається з даними?

- А. Приховування input автоматично стирає властивість моделі.
- Б. Blazor автоматично видаляє адресу з усіх старих замовлень.
- В. Прихована адреса обов'язково збережеться до БД.
- Г. Текст може залишитися в моделі, але сервіс зберігає порожню адресу для самовивозу.

8. Навіщо `PurchaseService` повторно виконує перевірку `CheckoutInput`?

- А. Щоб прискорити відображення CSS.
- Б. Щоб правила діяли й при виклику сервісу поза цією формою.
- В. Щоб автоматично виправити будь-яке неправильне введення.
- Г. Щоб не використовувати `RequestId`.

9. Чому фокус на полі імені встановлюється в `OnAfterRenderAsync` з перевіркою `firstRender`?

- А. Цей метод виконується до створення HTML-елементів.
- Б. Так фокус повертатиметься на ім'я після кожної зміни поля.
- В. Елемент уже доступний після інтерактивного рендерингу, а перевірка не дає повторно перехоплювати фокус.
- Г. Так браузер перестає перевіряти номер телефону.

10. Що означає `editContext.IsModified()` у цій роботі?

- А. Контекст має поля, позначені як редаговані; це не означає, що форма валідна.
- Б. Усі поля форми пройшли валідацію.
- В. Замовлення вже збережено в БД.
- Г. Поточні значення гарантовано відрізняються від початкових при точному порівнянні.

11. Навіщо в `Dispose` виконується `OnFieldChanged -= FieldChanged`?

- А. Щоб стерти замовлення з БД.
- Б. Щоб запустити перевірку форми повторно.
- В. Щоб змінити початкові значення полів.
- Г. Щоб прибрати раніше додану підписку на подію.

12. Що доводить проходження перевірки телефону регулярним виразом?

- А. Номер належить користувачу.
- Б. Рядок відповідає заданому формату, але існування й належність номера не підтверджені.
- В. На номер можна гарантовано надіслати повідомлення.
- Г. Покупець уже підтвердив замовлення телефоном.

Ключі відповідей і пояснення

№	Відповідь	Чому
1	В	Модель введення містить дозволені для редагування поля. Службові дані замовлення формує серверна логіка.
2	А	Значення належать моделі, а контекст пов’язує її зі станом полів і повідомленнями валідації.
3	Г	Формі задають або модель для створення контексту, або готовий контекст. Одночасне задання обох не допускається.
4	Б	Дочірній компонент редагує властивість спільного об’єкта. Збереження до БД — окрема операція сервісу.
5	В	Спочатку перевіряється форма. Успішна валідація дозволяє запустити обробник, але ще не гарантує успіху БД.
6	А	Адреса обов’язкова лише для доставки, тому правило аналізує кілька властивостей разом.
7	Г	Видалення поля з розмітки не очищує модель. Рішення про збережену адресу явно приймає сервіс.
8	Б	Сервіс можуть викликати тести, інша сторінка чи майбутній API. Він не повинен покладатися лише на поведінку UI.
9	В	Посилання на елемент доступне після рендерингу. `firstRender` обмежує встановлення фокуса першим відображенням екземпляра.

№	Відповідь	Чому
10	А	Ознака модифікації та валідність — різні характеристики. Повернення старого тексту також не обов’язково скидає цю ознаку.
11	Г	Відписування завершує зв’язок обробника з подією та відповідає попередньому `+=`.
12	Б	Регулярний вираз перевіряє синтаксис рядка, а не реальні обставини використання номера.